Web Server Fingerprinting Tool

1. Objective

The main objective of this project is to build a network scanning tool that can analyze a given website or IP address using both TCP and UDP protocols. The tool should be able to perform HTTP and HTTPS requests, extract server information, get SSL certificate details, and check if the system responds to UDP requests. It also compares the response time between HTTP and HTTPS.

2. Problem Statement

In networking, it is important to understand how different protocols work and how systems respond to them. While TCP-based services like HTTP and HTTPS are easy to test, UDP is harder because it does not always give a response. So, there is a need for a simple tool that can test both TCP and UDP communication and give meaningful results about the target system.

This project aims to create such a tool that can:

- Connect to a server using TCP and get details
- Probe UDP ports and check if there is any response
- Help understand how different protocols behave

3. Approach

The approach used in this project is straightforward.

First, the program takes input from the user in the form of a URL or IP address. Then it extracts the hostname and resolves it to an IP address.

For TCP:

- It connects to port 80 (HTTP) and sends a request
- It connects to port 443 (HTTPS) using SSL
- It reads the response and extracts server information
- It also collects SSL certificate details in case of HTTPS

For UDP:

- It sends packets to common UDP ports like 53 (DNS) and 123 (NTP)
- If a response is received, it means UDP is supported
- If not, it assumes the service is not responding or is filtered

The program also measures how much time HTTP and HTTPS requests take and compares them.

4. System Architecture

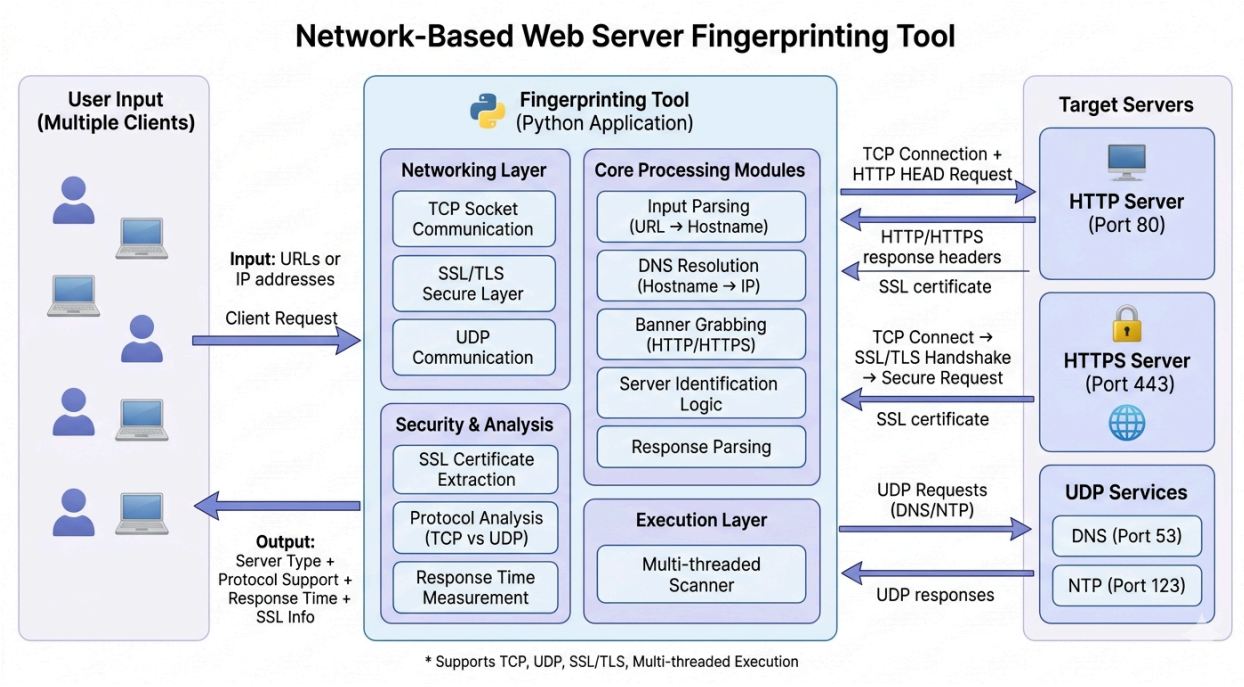
The system is divided into a few simple parts.

There is an input section where the user enters the target. Then the program parses this input and finds the hostname and IP address.

After that, two main operations happen:

- TCP scanning (for HTTP and HTTPS)
- UDP scanning (for selected ports)

Finally, all the results are processed and displayed together, including server details, certificate information, response times, and protocol analysis.



5. Technologies Used

The implementation was done using Python.

Socket programming was used to establish direct communication with servers.
SSL/TLS was used to handle secure HTTPS connections.
Threading was used so that multiple websites could be scanned at the same time without delay.

6. Implementation Details

This project is implemented in Python using basic libraries like `socket`, `ssl`, `threading`, and `time`.

The TCP part uses socket programming to connect to the server and send HTTP requests. The HTTPS part wraps the socket using SSL to establish a secure connection and then extracts certificate details.

The UDP part sends packets to specific ports and waits for a response. Since UDP is connectionless, it uses timeouts to decide whether a response is received or not.

Threading is used so that multiple targets can be scanned at the same time, which makes the program faster.

7. Setup and Execution

To run the project:

1. Make sure Python 3 is installed on your system
2. Save the program file as `fingerprinting.py`
3. Open Command Prompt / Terminal
4. Navigate to the folder where the file is saved
5. Run the command:
   ```
   python fingerprinting.py
   ```
6. Enter the list of URLs or IP addresses when prompted, separated by commas

Example:
```
google.com, github.com, 8.8.8.8
```

8. Sample Output

Target: 1.1.1.1 (1.1.1.1)

[TCP - HTTP]

Server: Server: cloudflare

Identified: Cloudflare

HTTP Time: 0.11 sec

[TCP - HTTPS]

Server: Server not found

Identified: Unknown

HTTPS Time: 0.16 sec

SSL Certificate:

No certificate

[UDP Scan]

Port 53: UDP ✔ (44 bytes, 0.03 sec)

Port 123: No response (timeout)

Protocol Analysis:

TCP: Supported

UDP: Supported (response received)

HTTPS slower than HTTP by 0.05 sec (TLS handshake overhead)

Total Time: 2.31 sec

9. Performance Analysis

From testing the program, a few observations were made.

HTTPS generally takes more time than HTTP. This is because HTTPS involves SSL/TLS handshake and encryption, which adds extra delay.

UDP responses are not always received. Some systems do not respond to UDP requests, which is normal behavior.

Using multithreading improves performance because multiple targets can be scanned at the same time instead of one by one.

10. Error Handling

Basic error handling is included in the program.

Timeouts are used to prevent the program from getting stuck when there is no response. Exceptions are handled for connection errors, invalid inputs, and SSL issues.

If a server does not respond or an error occurs, the program shows a proper message instead of crashing.

11. Conclusion

This project helped in gaining a clear understanding of how client-server communication works at a low level. By implementing socket programming directly, we were able to see how requests and responses are handled in real time.

The tool successfully identifies server software for different websites and demonstrates how fingerprinting can be done in a simple yet effective way.

12. Future Scope

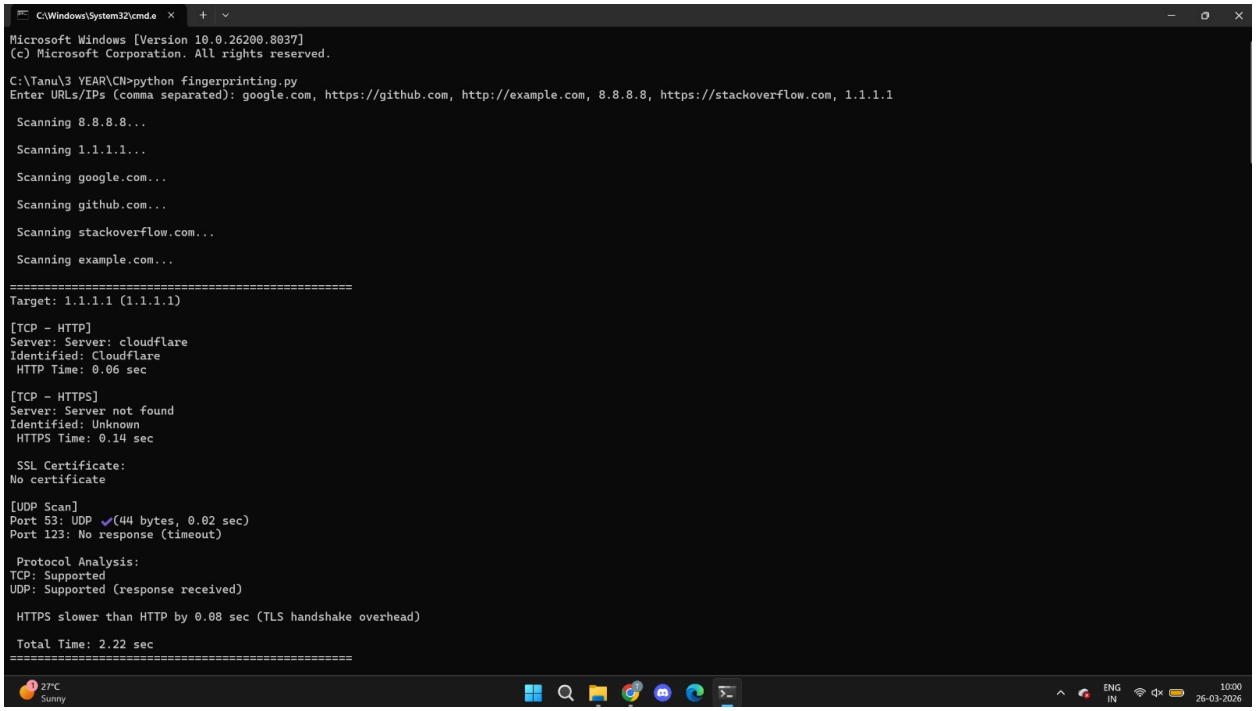
There are several ways this project can be extended in the future.

- More advanced techniques can be added to improve detection accuracy.
- Results can be stored in files for further analysis.
- A graphical interface can be developed to make the tool easier to use.
- Supporting newer protocols like HTTP/3 with external libraries

13. References

Python official documentation
Basic networking and HTTP protocol resources

14. Screenshots of Output



```
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Tanu\3 YEAR\CN>python fingerprinting.py
Enter URLs/IPs (comma separated): google.com, https://github.com, http://example.com, 8.8.8.8, https://stackoverflow.com, 1.1.1.1

 Scanning 8.8.8.8...

 Scanning 1.1.1.1...

 Scanning google.com...

 Scanning github.com...

 Scanning stackoverflow.com...

 Scanning example.com...

==================================================
Target: 1.1.1.1 (1.1.1.1)

[TCP - HTTP]
Server: Server: cloudflare
Identified: Cloudflare
 HTTP Time: 0.06 sec

[TCP - HTTPS]
Server: Server not found
Identified: Unknown
 HTTPS Time: 0.14 sec

 SSL Certificate:
No certificate

[UDP Scan]
Port 53: UDP ✔(44 bytes, 0.02 sec)
Port 123: No response (timeout)

 Protocol Analysis:
TCP: Supported
UDP: Supported (response received)

 HTTPS slower than HTTP by 0.08 sec (TLS handshake overhead)

 Total Time: 2.22 sec
==================================================
```

```
C:\Windows\System32\cmd.e

Target: 8.8.8.8 (8.8.8.8)

[TCP - HTTP]
Server: Server not found
Identified: Unknown
 HTTP Time: 2.0 sec

[TCP - HTTPS]
Server: Server: HTTP server (unknown)
Identified: Unknown
 HTTPS Time: 0.11 sec

 SSL Certificate:
Issued To: dns.google
Issued By: WR2
Valid From: Mar  9 08:38:27 2026 GMT
Valid Till: Jun  1 08:38:26 2026 GMT

[UDP Scan]
Port 53: UDP ✔(44 bytes, 0.02 sec)
Port 123: No response (timeout)

 Protocol Analysis:
TCP: Supported
UDP: Supported (response received)

 HTTP slower than HTTPS by 1.89 sec

 Total Time: 4.13 sec
==================================================

==================================================
Target: github.com (20.207.73.82)

[TCP - HTTP]
Server: Server not found
Identified: Unknown
 HTTP Time: 0.05 sec

[TCP - HTTPS]
Server: Server: github.com
Identified: Unknown
 HTTPS Time: 0.2 sec

 SSL Certificate:
Issued To: github.com
```

```
C:\Windows\System32\cmd.e

[TCP - HTTPS]
Server: Server: github.com
Identified: Unknown
 HTTPS Time: 0.2 sec

 SSL Certificate:
Issued To: github.com
Issued By: Sectigo Public Server Authentication CA DV E36
Valid From: Mar  6 00:00:00 2026 GMT
Valid Till: Jun  3 23:59:59 2026 GMT

[UDP Scan]
Port 53: No response (timeout)
Port 123: No response (timeout)

 Protocol Analysis:
TCP: Supported
UDP: Not confirmed / filtered

 HTTPS slower than HTTP by 0.15 sec (TLS handshake overhead)

 Total Time: 4.26 sec
==================================================

==================================================
Target: example.com (104.18.26.120)

[TCP - HTTP]
Server: Server: cloudflare
Identified: Cloudflare
 HTTP Time: 0.08 sec

[TCP - HTTPS]
Server: Server: cloudflare
Identified: Cloudflare
 HTTPS Time: 0.18 sec

 SSL Certificate:
Issued To: example.com
Issued By: Cloudflare TLS Issuing ECC CA 3
Valid From: Feb 13 18:53:48 2026 GMT
Valid Till: May 14 18:57:50 2026 GMT

[UDP Scan]
Port 53: No response (timeout)
Port 123: No response (timeout)
```

```
Port 53: No response (timeout)
Port 123: No response (timeout)

 Protocol Analysis:
TCP: Supported
UDP: Not confirmed / filtered

 HTTPS slower than HTTP by 0.1 sec (TLS handshake overhead)

 Total Time: 4.26 sec
==================================================

==================================================
Target: stackoverflow.com (198.252.206.1)

[TCP - HTTP]
Server: Server: cloudflare
Identified: Cloudflare
 HTTP Time: 0.16 sec

[TCP - HTTPS]
Server: Server: cloudflare
Identified: Cloudflare
 HTTPS Time: 0.35 sec

 SSL Certificate:
Issued To: stackoverflow.com
Issued By: E8
Valid From: Feb 19 14:15:02 2026 GMT
Valid Till: May 20 14:15:01 2026 GMT

[UDP Scan]
Port 53: No response (timeout)
Port 123: No response (timeout)

 Protocol Analysis:
TCP: Supported
UDP: Not confirmed / filtered

 HTTPS slower than HTTP by 0.19 sec (TLS handshake overhead)

 Total Time: 4.52 sec
==================================================

==================================================
Target: google.com (142.250.67.46)
```

```
C:\Windows\System32\cmd.e
==================================================
Target: google.com (142.250.67.46)

[TCP - HTTP]
Server: Server: gws
Identified: Google Web Server
 HTTP Time: 0.39 sec

[TCP - HTTPS]
Server: Server: gws
Identified: Google Web Server
 HTTPS Time: 0.46 sec

 SSL Certificate:
Issued To: *.google.com
Issued By: WE2
Valid From: Mar  9 08:36:27 2026 GMT
Valid Till: Jun  1 08:36:26 2026 GMT

[UDP Scan]
Port 53: No response (timeout)
Port 123: No response (timeout)

 Protocol Analysis:
TCP: Supported
UDP: Not confirmed / filtered

 HTTPS slower than HTTP by 0.07 sec (TLS handshake overhead)

 Total Time: 4.86 sec
==================================================

 Scan Completed

C:\Tanu\3 YEAR\CN>
```

15. Code

```
import socket
import ssl
import threading
import time
from urllib.parse import urlparse

TIMEOUT = 2
lock = threading.Lock()

def parse_target(target):
    if not target.startswith("http"):
        target = "http://" + target
    return urlparse(target).hostname



def resolve_ip(host):
    try:
        return socket.gethostbyname(host)
    except:
        return "Resolution Failed"



def receive_full_data(sock):
    data = b""
    sock.settimeout(TIMEOUT)

    try:
        while True:
            chunk = sock.recv(4096)
            if not chunk:
                break
            data += chunk
    except socket.timeout:
        pass

    return data.decode(errors="ignore")


def tcp_http(host):
    try:
```

```
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(TIMEOUT)
        s.connect((host, 80))

        req = f"HEAD / HTTP/1.1\r\nHost: {host}\r\nConnection: close\r\n\r\n"
        s.send(req.encode())

        res = receive_full_data(s)
        s.close()
        return res
    except Exception as e:
        return f"Error: {e}"



def tcp_https(host):
    try:
        context = ssl.create_default_context()

        # Step 1: raw TCP connect (no TLS yet)
        raw = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        raw.settimeout(TIMEOUT)
        raw.connect((host, 443))         # <-- TCP connect on raw socket

        # Step 2: wrap for TLS (handshake happens here)
        secure = context.wrap_socket(raw, server_hostname=host)
        secure.settimeout(TIMEOUT)

        cert = secure.getpeercert()

        req = f"HEAD / HTTP/1.1\r\nHost: {host}\r\nConnection: close\r\n\r\n"
        secure.send(req.encode())

        res = receive_full_data(secure)
        secure.close()

        return res, cert

    except Exception as e:
        return f"Error: {e}", None



def udp_scan(host):
```

```
    results = []
    ports = [53, 123]  # DNS, NTP

    for port in ports:
        try:
            sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
            sock.settimeout(TIMEOUT)

            start = time.time()

            if port == 53:
                # Build a minimal DNS query for "google.com" regardless of
                # whether the target is a hostname or a bare IP like 8.8.8.8.
                query = b'\xaa\xaa\x01\x00\x00\x01\x00\x00\x00\x00\x00\x00'
                domain = "google.com"
                for part in domain.split('.'):
                    query += bytes([len(part)]) + part.encode()
                query += b'\x00\x00\x01\x00\x01'
                sock.sendto(query, (host, port))

            elif port == 123:
                # NTP request
                sock.sendto(b'\x1b' + 47 * b'\0', (host, port))

            try:
                data, _ = sock.recvfrom(512)
                end = time.time()
                results.append(
                    f"Port {port}: UDP ✔ ({len(data)} bytes, {round(end - start, 2)} sec)"
                )
            except socket.timeout:
                results.append(f"Port {port}: No response (timeout)")

            sock.close()

        except Exception as e:
            results.append(f"Port {port}: Error ({e})")

    return results



def extract_server(response):
    if not isinstance(response, str):
```

```
        return "Invalid"

    for line in response.split("\r\n"):
        if line.lower().startswith("server:"):
            return line.strip()

    if "cloudflare" in response.lower():
        return "Server: Cloudflare (hidden)"

    return "Server not found"


def identify(response):
    if not isinstance(response, str):
        return "Unknown"

    r = response.lower()

    if "apache" in r:
        return "Apache"
    if "nginx" in r:
        return "Nginx"
    if "iis" in r:
        return "Microsoft IIS"
    if "cloudflare" in r:
        return "Cloudflare"
    if "gws" in r:
        return "Google Web Server"

    return "Unknown"

def format_cert(cert):
    if not cert:
        return "No certificate"

    try:
        subject = dict(x[0] for x in cert.get("subject", []))
        issuer = dict(x[0] for x in cert.get("issuer", []))

        return (
            f"Issued To: {subject.get('commonName', 'N/A')}\n"
            f"Issued By: {issuer.get('commonName', 'N/A')}\n"
            f"Valid From: {cert.get('notBefore', 'N/A')}\n"
            f"Valid Till: {cert.get('notAfter', 'N/A')}"
```

```
        )
    except:
        return "Certificate parsing error"


def scan(target):
    host = parse_target(target)
    ip = resolve_ip(host)

    print(f"\n Scanning {host}...")

    start_total = time.time()

    # HTTP
    h_start = time.time()
    http_res = tcp_http(host)
    h_time = round(time.time() - h_start, 2)

    # HTTPS
    s_start = time.time()
    https_res, cert = tcp_https(host)
    s_time = round(time.time() - s_start, 2)

    # UDP
    udp_results = udp_scan(host)

    end_total = time.time()

    with lock:
        print("\n" + "=" * 50)
        print(f"Target: {host} ({ip})")

        print("\n[TCP - HTTP]")
        print("Server:", extract_server(http_res))
        print("Identified:", identify(http_res))
        print(f" HTTP Time: {h_time} sec")

        print("\n[TCP - HTTPS]")
        print("Server:", extract_server(https_res))
        print("Identified:", identify(https_res))
        print(f" HTTPS Time: {s_time} sec")

        print("\n SSL Certificate:")
        print(format_cert(cert))
```

```
        print("\n[UDP Scan]")
        udp_supported = False
        for r in udp_results:
            print(r)
            if "✔" in r:
                udp_supported = True

        print("\n Protocol Analysis:")
        print("TCP: Supported")

        if udp_supported:
            print("UDP: Supported (response received)")
        else:
            print("UDP: Not confirmed / filtered")

        diff = round(s_time - h_time, 2)
        if diff >= 0:
            print(f"\n HTTPS slower than HTTP by {diff} sec (TLS handshake overhead)")
        else:
            print(f"\n HTTP slower than HTTPS by {abs(diff)} sec")

        print(f"\n Total Time: {round(end_total - start_total, 2)} sec")
        print("=" * 50)

def main():
    user_input = input("Enter URLs/IPs (comma separated): ")
    targets = [t.strip() for t in user_input.split(",")]

    threads = []

    for t in targets:
        th = threading.Thread(target=scan, args=(t,))
        threads.append(th)
        th.start()

    for th in threads:
        th.join()

    print("\n Scan Completed")


if __name__ == "__main__":
    main()
```